LC 621 Task Scheduler：从构造、对齐到最优公式

完整推导与 Debug 全记录（大白话版） · V6 · 2026-10 · Bo Hou 与 ChatGPT 协作整理

本文要回答的不是「公式是什么」，而是「这个公式为什么从构造里出现」。尤其记录 Bo 提出的按频次排序、每行 n+1 个槽、相邻行同任务同列对齐、同频 tie-breaking 保留原始 top-k、曾考虑但最终放弃的置换假设，以及 k 的边界，以及助手在讨论中的反复误判。

阅读约定：「Bo 的提出」是讨论中用户自己的想法；「助手误判」单列说明；「已证」与「待证」严格分开。算法正确与某段对话中的证明正确，不是同一件事。

0. 一页路线图：最终得到什么、过程为什么绕

阶段	真正提出的问题	本篇位置
最初单堆	最早可执行 vs 剩余次数最多，是否一样？	第 2 节
骨架填空	U1 固定、x 剩余，Ui+1 放不下怎么办？	第 3 – 4 节
结构突破	同一个调度，为什么拉大 A 间距反而更好？	第 6 节
统一构造	每行 n+1，重算频次，相邻行相同任务同列	第 7 – 10 节
数学化	骨架下界 + 总任务数量；充分性到底怎么证明？	第 11 – 13 节
工程落实	Heap 模拟、直接公式、代码与自测	第 14 – 15 节

1. 原题、记号与必须严守的逻辑

任务都是一个时间单位，同一种任务相邻两次执行必须间隔至少 n 个其它时间槽；可调换执行顺序，可 idle；只要求最少总时间。

```
N = 所有 task 的总执行次数
U1 >= U2 >= ... >= Um = 各种任务的频次（从大到小）
f = U1,  k = 与 f 并列最高频的 task 类型数
L = n + 1 = 同类任务最小的执行位置差
```

特别区分三个不同命题：（1）任何合法调度不能短于某个值，这是下界；（2）能画出一个合法排列，这是一个上界；（3）二者相等，才算证明最优。讨论中很多来回都是因为把其中两步混为一谈。

Bo 使用的 U_i 是按频次排序后的第 i 种 task 的频次，不是任意的任务排列；这一点曾经被助手多次忽略。

2. Round 1：最初的「一个堆」方案

「task -> unit number，用 (avail, remain) 小顶堆，弹出后执行一次、减少剩余次数，更新下次可执行时间。」

这套方案能保证按冷却时间合法运行，但若比较器先按最早可执行时间（avail）排序，可能优先执行很早就解禁但剩余次数少的任务，导致多余 idle。

```
tasks:  A  A  A  B  C,  n=1
单堆  (avail,  -remain)  可能得到:  A  B  C  A  _  A  (6)
更短的合法排列:                                A  B  A  C  A  (5)
```

反例指出的是选择优先级错误，不是「Heap 一定不行」。Bo 随后指出从当前可执行任务里优先选择剩余次数最多的任务，应当区分「资格（avail）」与「优先级（remain）」。

回顾：助手曾用不构成反例的排列来反驳单堆，再被 Bo
要求重新检查。这些反复核对推动了最优性讨论，但不应被写成一次就完成的推导。

3. Round 2：Bo 的 U1 骨架，不是泛泛而谈的填空

「先安排最大的 U1，再 U2，再 U3；空位不够就看 Ui+1。」

如果最大任务 A 出现 f 次，那么按最短允许间隔放置：A 间隔 n 个空位；按 1-based 位置，最后一个 A 的最早位置是 (f-1)L+1。注意此处应当是 (f-1)(n+1)+1，不是 U1*n+1：讨论中曾混淆「中间空位 n」与「相邻执行位置差 n+1」。

```
A  x4,  n=2,  L=3
位置    1  2  3  4  5  6  7  8  9  10
任务    A  _  _  A  _  _  A  _  _  A
共有三段各两个内部空位  =  (f-1)*n  =  6
```

Bo 的容量变量 x：假设按频次顺序填入一些类别，x 表示特定构造方式中尚未使用的有效空位。讨论中写过 x=U1-(U2+...+Ui)，它可以描述某种按列分配的剩余「轮次资源」，但不等于全部时间槽容量 (f-1)*n。必须先声明 x 的单位与其所在构造，否则任何增量公式都会混淆。

4. 为什么「装不下就接在后面」会让证明卡死

原策略是：保留已经安排的前缀，把 Ui+1 能塞的部分塞进去，剩余 Ui+1-x 和后续类别重新排序，接在 U1 最早结束时刻后面。它能生成候选调度，但局部位置一旦固定，后续可能产生多余等待：单种任务的最早完成时间 ≠ 全部任务的最早完成时间。

Bo 反复指出这不是任务总数相加就能解决的问题，具体空位分布、同类之间最小间隔、跨段衔接都要同时处理。这使构造自然陷入 alignment（对齐）与 constrained packing 的分类讨论。

助手当时反复说「不能保证最优」，却没有给出精确反例，还多次忽略 U_i
已排序的前提，导致推导停滞。关键教训：否定一个候选定理，必须指出它的量纲、缺失前提或真正的反例。

4A. 失败的不变量搜索日志：最重要的数学 Debug

上一版在这里直接跳到“ 追加不可保证最优 ”，漏掉了推导过程中最有价值的一段：Bo 先后提出了若干候选不变量，随后又亲自识别出其中的不成立条件。这不是一次简单的推导失败，而是对状态能否压缩、子问题能否独立、贪心操作能否保持最优性的连续审计。以下按讨论顺序记录。

4A.1 尝试一：最早完成每一种 task，可否合成全局最早完成？

> 「对于任意一个 task，这样子调度是这个 task 完成的最小时间；所以每个 task 都这样调度。但是不同 task 之间怎么安排，最优还需要证明。」

候选性质：每一种 task 按最小冷却间隔完成，整体就容易最优。Bo 自己明确发现的漏洞：各 task 竞争同一 CPU 槽位，单个 task
的最早完成时间并不具备直接可加性；还必须解决它们之间的位置协调。此处没有建立出全局不变量。

这一步非常重要：不是助手告诉 Bo “ 构造不够 ”，而是 Bo 自己把“ 单 task 的局部最小 ”与“ 多 task 的全局最小 ”分开了。后面所有 alignment 困难都从这里暴露。

4A.2 尝试二：按频次排序后，固定已安排的位置（前缀固定不变量）

> 「先放最大的 U1，再 U2，然后不够放了，看怎么 extend。」

候选不变量：按 U1>=U2>=... 的顺序填入后，前面已经安排好的 job 位置尽可能保持不动；新放入的任务不影响已有前缀的合法性与最优性。问题：固定前缀确实可以帮助维持局部合法性，但它把后续优化限制在越来越小的空位集合。是否能保持全局最优，不能从“ 之前的位置合法 ”推出。

Bo 随后反复强调：这个方向并非完全错误，而是追求最小完成时间时容易被“ 先固定再对齐 ”吸住，无法有效分析剩余容量不足的情况。

4A.3 尝试三：引入 x，写成减小后的同型子问题

> 「x = U1 - (U2 + ... + Ui)；当 Ui+1 不够放，剩余 Ui+1 - x，和 Ui+2 ...
> 重新排序，然后从原骨架后面接着安排。」

候选不变量：已用 x 个容量填入一部分 Ui+1 后，剩余问题仍具有同样的“ 频次排序 + 骨架填空 ”形式；因此可以递归或迭代处理。这是典型的“ 状态缩减后结构不变 ”设想。

Bo 自己指出的缺口：剩余的 Ui+1-x 不是一个与前半段无关的全新任务类型，它与前缀中已执行的同类任务仍有冷却约束；并且先前的任务位置固定住，可能妨碍调整整体最优安排。只按剩余次数重新排序，不能自动得到一个等价的子问题。

> 「我自己意识到 Ui+1-x 不是可以的循环不变性……这个等价性不一定成立。」

这是一次明确的自我纠错：Bo 没有把 x 当作“ 不变量已经存在 ”的证据，而是追问状态里遗漏了什么。上一版把它简写为“ 局部固定未证最优 ”，丢掉了这个推理价值。

4A.4 尝试四：每一段都最优，拼接是否最优？

「如果每一段都是最优，为啥不是呢？」

候选性质：先在初始骨架内找到最优填法，再在后续区域按剩余频次安排，段段最优可以合成为全局最优。问题：分段子问题之间具有跨段冷却依赖，且第一段选定的边界状态会影响第二段可行解空间；“ 每段最优 ” 并不天然推出 “ 拼接最优 ”。

这和 4A.3 不是重复：4A.3
问的是 “ 剩余问题是否同型 ”，此处问的是 “ 若各段都已最优，目标值是否能合成 ”。Bo
对两个层面的质疑都提出过。

4A.5 尝试五：把剩余 Ui+1-x 直接当作最小增量

「从 U1*n+1 之后继续安排 Ui+1-x；这里还是不对劲。」

候选量：初始骨架的最后时刻 + 尚未安放的 task 个数。问题：未安放的执行次数是额外占用槽位的数量，但并不自动等于必须额外增加的完成时间；如果允许前面任务的实际间隔扩大，可能重新分布已有槽位并减少尾部等待。

这里还有一次术语误解：Bo 强调说的是 U1*n+1（当时自用的时间编号表达），助手误当作 U1*(n+1) 并批评 “ 重复计数 ”；后来澄清讨论焦点在于时间接续和全局重排，而不是机械加法。技术博客须如实保留这一分歧，不能悄悄改写成助手从一开始就讲清楚了。

4A.6 尝试六：总容量足够，是否自动可行？

「U1*n > U2+...+Ui，不代表 2 到 i 都可以填进去，还有冷却性的限制。」

候选不变量：总剩余空位数足以容纳任务次数时，后续总能完成填充。Bo
自己否定的关键：总量条件只计数，没有计入同类间隔与空位几何分布；容量不等于可行性。这使 “ 只看 x ” 的压缩状态再次不足。

随后 Bo 明确指出：如果继续沿每一类 task 的 alignment 分类讨论，会落入类似 constrained packing 的复杂分支；他选择暂时离开这个思维吸引子，寻找更抽象、可以对全局证明的结构。

4A.7 尝试七：能否建立逐轮选择保持的循环不变量？

「这个好像也没有直接得到 loop invariant，因为后续得 align。」

候选不变量：当前行或当前前缀合法、最高频任务按规则放好，下一次贪心填入仍保持 “ 某种最优前缀 ”。失败原因：合法性是局部可验证的，但 “ 最优前缀 ” 需要定义对未来剩余频次与冷却状态的充分信息；在固定 U1、逐类别填充的框架内，未来任务位置始终会反过来影响此前的安排是否为全局最优。

这是 Bo 从 “ 找一个具体排法 ” 上升到 “ 什么状态才能被证明不变 ” 的节点。重要的是：Bo
主动识别循环不变量缺失，而不是在编程实现以后才发现错误。

4A.8 尝试八：同频追平后的 “ 置换 ” 猜想，后来被更简单的不变量替代

「A 减一后与 D 追平，类似置换；最后留下的是某种置换的隐式 k 个，而不是一开始固定身份的 k 个。」

这是探索过程中的临时猜想，而不是最终结论。Bo 曾设想用同频时的“ 隐式 k
个角色 ”来解释剩余频次的演化；随后在 k<L
的情况中进一步发现：根本不需要追踪被置换的隐式身份。初始最高频的 top-k 个任务可以通过 tie-breaking
优先保留，连续每行各执行一次，直到完成。

关键不变量：只要这 k 种任务还有剩余，每行都先给它们预留 k
个槽位；它们每轮同步减一，其他任务初始频次不高于它们；其他任务追平时，仍优先选原始
top-k，不必替换其身份。剩余 L-k 个槽位再选其它高频任务。

因此，早先“ 追平后必须发生置换 ”“ 最后只剩某种隐式 k ”应明确标注为已放弃的证明路线。这里只针对 k<L
的保护性质；k>=L 时不能让最初的 k 种任务每行全部执行，须另行分析轮流入行，不能偷换前提。

4A.9 失败尝试总表：从“ 局部合法 ”转向“ 全局充分条件 ”

尝试	想要保持什么	Bo 或审计发现的问题	状态
单 task 最早	每种 task 都尽早结束	CPU 槽位竞争；局部最优不能相加	放弃作为全局证据
固定前缀	已安排位置不动仍最优	会约束后续重排	仅保留合法性
x 缩减	Ui+1-x 仍是同型子题	缺少冷却边界状态	Bo 主动识别失败
分段拼接	每段最优可直接合成	跨段依赖未消失	没有组合性证明
最小增量	剩余数量直接等于延长量	重新排列可能改变尾部等待	未成立
容量充分	数量够就能塞入	同类冷却 / 位置分布	Bo 主动指出漏洞
逐项循环不变量	每次贪心保持最优前缀	alignment 使状态不闭合	改换构造表达
隐式 k / 置换	猜想同频时必须交换角色	k<L 可固定原始 top-k 并 tie-break 保护	已放弃置换假设

这一串尝试不是“ 绕远路 ”。 它们逐步定位出真正需要补进状态里的信息：不能只记录频次和剩余容量，还必须让调度位置的冷却结构可控。随后 Bo 才提出每行
L=n+1、相邻行同任务同列的局部可证规则。这个转折与后面的公式推导之间存在实质联系。

4B. Lesson：Alignment
思维吸引子——为什么要主动跳出一种正确但低效的证明框架

这段不是“我没想出来”式的总结，而是 Bo 在讨论中主动提出的研究方法论：当固定 U1、逐类填充的推导，每次都被拖回“Ui+1 怎么对齐”时，说明当前表述可能陷入思维吸引子。此时再多处理几个排列特例，未必能让最优性证明前进一步。

> 「这个证明方式不自己会滑向这个吸引子……感觉肯定哪里我没 capture 住。」

> 「所以这个不对，应该再思考远离这个思维的吸引子。」

吸引子是怎么形成的：（1）先用 U1 的最小完成时间定骨架；（2）为了维持已经安排的“局部最优”，固定已有位置；（3）填入 Ui+1 时出现容量、冷却与尾部追加；（4）为修复它引入 x、再排序或延长时间；（5）下一轮又遭遇新的 alignment。每一次局部修补，都默认了“前面固定是必要的”，于是越来越难审查这个默认假设。

Bo 当时真正反思的不是 U1 信息是否有用。最大频次与冷却约束当然有用；他质疑的是“必须固定最高频任务的绝对位置、并按任务类型逐项填空”的证明表达方式。这两者不能混淆。

为什么这叫方法论 lesson：一个解题方向可行，不代表它适合进行形式化证明；特别是当候选循环不变量不断要求记住更多过去的排列细节时，可能意味着状态压缩方式不对。合适的做法不是宣布原方向错误，也不是机械换一道例子，而是明确列出不变量失败的原因，再改写构造时维护的量。

这次如何跳出来：后续提出“每行 n+1、按剩余频次选择、相邻行同任务同列”。在这个表述里，alignment 从“对每个 Ui+1 重新处理大量位置组合”变成“相邻行共有任务保持同列”的局部性质。它没有凭空删除冷却约束，而是改变了证明中追踪的结构。

关于协作的教训：助手多次笼统说“还没有证明最优”“尝试另一种方向”，却没有首先核验 Bo 已经明确的排序、x 和构造约束。这种反馈本身也形成了讨论上的吸引子：不断回到同一个抽象质疑，不产生新信息。有效的 Review 必须指出精确的未证命题或反例，不能反复重述结论。

可复用的检查清单：陷入某个局部 case 时，先问：固定位置是题目约束还是人为选择？现有状态是否遗漏跨段关系？“前缀局部最优”能否推出全局最优？能否换一种构造，使困难的全局约束变成可检验的局部不变量？

5. 真正的转折：不把 A 的「最早完成」当全局目标

13 槽构造使 B/C/D 很早完成，最后剩下 E 需要等冷却；11 槽构造让 A、B、C、D、E 的执行分布更均匀，反而无 idle。Bo 正确地区分：最小化每一种任务各自的最后完成时刻，不等于最小化全体任务结束时刻。

于是改用按剩余频次驱动的调度，而不是固定 U1 所有位置。这个转折不是放弃 U1 统计量，而是放弃「必须固定它的具体绝对位置」这一额外假设。

6. Round 4：Bo 的统一分行构造——本篇核心

这正是上一版写得太少的内容。一行长度为 L=n+1。每一轮，从当前剩余次数最多的任务类型里选出最多 L 种不同 task，每种放一次；执行后频次减一，再决定下一行。相同任务在相邻两行都出现时，同列对齐。最后一行可以不足 L，若因冷却不得不等则填 idle。

```
规则（候选构造）
1) count[type] = 剩余次数；每行挑不重复的类型，优先 count 高。
2) 选中的每个类型 count--；完成的类型不再参与。
3) 与上一行重复的类型，保持同一列。
4) 只出现在本行的新类型，放剩余列；最后一行尽量提早结束。
5) 若行没满但尚有重复任务，需说明为何填 idle 必要。
```

前三步是确定可验证的合法性机制。第 4 – 5 步涉及最佳的 tie-breaking、跨行兼容性与最优性，不能仅由一个例子直接断言一般成立。

7. 为什么同列对齐有效？把 alignment 化成局部不变量

设相邻两行各长 L。两行都出现的 task 放在同一列 j，则实际执行位置差为 L，刚好满足最小间隔 n=L-1。若跨过一整行再次出现，即使前次在行末、本次在下下行行首，间距至少 L+1，必然合法。

```
L=4（n=3）
上一行: A B C D
下一行: A E C F <- A在第1列，C在第3列
位置差：A = 4, C = 4，都是合法的。
```

Bo 强调：「上一行确定后，下一行共有 task 的列位置就确定；其余放空列」。对两行的固定任务集合，这个说法成立。需要额外小心的是连续多行同时保持的列约束：如果某个任务在两行相邻出现就被固定在一列，而另一个任务后来追平频次进入，必须选择一套 tie-breaking 使列空间相容。

8. k 小于行宽：top-k 保护不变量，不需要置换

若初始恰有 k 种最高频次任务，k<L，则每行有至少 L-k 个其它槽位。Bo 提出：同频时 tie-breaking 优先保护这 k 个最高频任务，使其每行都能被选中，固定相应列；其它已追平的任务用剩余位置进入，被减一后再重新排序。

```
n=3, L=4
A x4, B x4, C x2
行1: A B C _
行2: A B C _
行3: A B _ _
行4: A B 最后无需补满
完成时刻: 14
```

这个例子有 idle：同列保护 A/B，剩余任务消耗以后每轮空出槽位。它与骨架主导情况一致。但「原先 k 个任务每轮都被选中」必须在给定的选择与 tie-breaking 规则下证明；不能因为 k<L 就跳过具体选择过程。

讨论曾试过一种更复杂的“ 置换 ”理解：当 A 与 D 的剩余频次追平，让 D 接替 A 的高频角色。但 Bo 后来明确指出：这一步完全不必要。当 k<L 时，tie-breaking 始终优先保留原始 top-k，其他追平的任务从其余 L-k

个槽位进入。无需定义抽象的“隐式 k”，也不需要证明角色置换。

9. k 大于行宽：同一个规则，不需要另起算法

当 k>=L 时，最多 L 个位置的行容不下全部 k
个最高频类型，于是按剩余频次逐行轮流选；不要求初始最高频类型每行全部出现。Bo
坚持：规则根本没改，只是最高频集合超过行容量后的自然演化。

```
n=2,  L=3,  A/B/C/D/E  各  x3，N=15,  k=5
行1:  A  B  C
行2:  D  E  C
行3:  D  E  A
行4:  B  C  A
行5:  B  D  E
展平：A  B  C  D  E  C  D  E  A  B  C  A  B  D  E
```

该排列相邻行重复的 C（第1/2行第3列）、D/E（第2/3行第1/2列）、A（第3/4行第3列）、B（第4/5行第1列）满足同列规则；其余重复任务跨行间隔足够。因此整个 15 槽排列合法且无 idle。

请注意这里的 k=5>L=3；按行划分时最高频类型会隔行出现。这回答了为什么初始 k
不一定能被完整安放在单行里，也说明把「最大频次骨架」想成每行固定一组 task 会遇到困难。

10. 「有 idle / 无 idle」能否用同一个构造证明？

Bo 的目标是统一论证：不论有无 idle，都按相同的剩余频次优先与分行规则构造。 若排满全部 N 个时间槽不需要 idle，则最短时间为 N；否则某些非末行槽位不可避免空闲，时间由最高频冷却的骨架控制。

对于 k<L 的 idle 主导情形，Bo 最终用更直接的 top-k 保护性质替代了早先的“隐式置换”设想：这 k
种任务持续参与各行、同步减一；当其他任务追平时，用 tie-breaking 保护它们，而不是换人。这样最后的 k
不再是一个无法定义的“置换后 k”，而是原始最高频的 k。此处仍需区别：top-k
保留性质与完整调度最优性的形式化证明不是同一个命题。

助手曾错误地拿无 idle 的例子反驳只针对 idle
情形的命题，后来撤回。检验一个命题必须保留其适用条件——这是这段 back-and-forth 的重要教训。

11. 两个下界从哪里来，不是靠摆漂亮的例子

第一个下界很直观：所有 N 个任务必须各占一个时间槽，所以
T>=N。第二个下界来自最高频任务所需的相互间隔。如果最高频为 f，且并列最高频有 k 类，前 f-1
次的重复轮次之间至少需要 n+1 的间距，末轮还要容纳这 k 次执行。

```
冷却下界  F  =  (f-1)*(n+1)  +  k
任务下界  N  =  sum(count[type])
```

```
任意合法调度:  T  >=  max(N,  F)
```

真正不能跳过的是反方向：是否一定能构造长度恰好 max(N,F) 的调度？Bo
花了最多时间研究的，就是这一条充分性证明。

12. 补足正式证明：一种完全可机械执行的存在性构造

为了不把第 7 – 11 节尚未闭合的「同列+逐行最大频次」强行写成定理，这里单独给出一条可验证的存在性证明思路：允许使用不同于对话中原始分行贪心的构造；明确区分它是补充证明，不冒充 Bo 原本提出的规则。

令 T=max(N,F)，定义一个长度 T 的时间表。有 T-N 个 idle 占位符。选择长度为 L=n+1 的轮次，并把每类 task 的多次出现尽量放在不同轮次。所有最高频 task 在前 f-1 个轮次中占相同的相对序位，最后轮次执行 k 种最高频 task。其余低频任务填充不同轮次；当剩余任务足以覆盖空位时，可将轮次边界向后移动，直至没有 idle。

这提供了证明的整体结构，但「可以将轮次边界移动且不破坏所有冷却约束」仍需形式化交换论证。为避免再重犯逻辑跳跃，本篇不把这段文字标为完整严格证明。如果目标是论文级形式化，可以进一步单独建立并证明一个调度存在性引理；LeetCode 的算法公式是已知正确结论，正文主要真实记录推导路径。

13. 可执行的 Heap 模拟：与公式解法的差别

工程上，要输出实际执行序列，可以用两个队列：max-heap 存可执行任务的剩余次数，min-heap 存冷却中任务的解禁时间。任何时刻把已解禁任务全部移回
max-heap，执行剩余次数最多的类型；若没有可执行任务，就跳到最近一次解禁。单纯按 (avail,-remain) 排序不行，因为 avail 只表示资格。

```
struct  Cool  {  int  ready,  left,  type;  };
//  ready  表示下一次允许开始执行的时间
//  执行  task  于  time=t，则  ready=t+n+1
//  available:  max-heap  by  left;  cooling:  min-heap  by  ready
```

Heap
调度可以帮助观察贪心的实际演化，但正确性仍需要证明；公式法只输出长度，因此不需要模拟每一个时间槽。

14. 最终 C++ 与复杂度

```
int  leastInterval(vector<char>&  tasks,  int  n)  {
        int  cnt[26]  =  {0};
        for  (char  c  :  tasks)  ++cnt[c  -  'A'];
        int  f  =  0;
        for  (int  x  :  cnt)  f  =  max(f,  x);
        int  k  =  0;
        for  (int  x  :  cnt)  if  (x  ==  f)  ++k;
```

```
        return max((int)tasks.size(), (f - 1) * (n + 1) + k);
    }
```

任务类型固定为 26，时间 O(N+26)=O(N)；额外空间 O(26)=O(1)。若任务类型没有固定字母表，空间则依赖不同 task 类别数量。

15. 讨论复盘：哪些是推进，哪些是浪费

讨论片段	实际价值 / 问题
Bo 的单 Heap	识别 avail 与 remain 排序并非同一件事；错误策略可以被反例推翻。
固定 U1 与 x	抓住了密度/容量直觉，但逐类追加会造成 alignment 技术债。
13 与 11 槽排列	清楚地暴露「合法不等于最优」以及具体位置固定的局限。
每行 L、相邻同列	Bo 提出的核心构造；把大量排列约束降到跨行的局部合法性。
同频 tie-breaking	早期置换猜想已放弃；k<L 时保留原始 top-k 并让它们每轮同步减一。
k<L / k>=L	同一个选择机制的两类频次演化；不是两套算法。
助手多次误判	忽略用户定义、未核对位置、用无关反例否定有条件命题、过早给公式。
尚需单独形式化	一般输入中，分行选择、同列约束与无 idle 最优性如何同时保持。

16. 最终 takeaway：读这题应该留下的内功

（1）下界容易猜，难点是可达性；（2）单个频繁任务的最早完成时刻不是全局目标；（3）同频追平不强制任务置换；k<L 时可用 tie-breaking 保留原始 top-k；（4）把调度问题写成 L=n+1 的行，可以清楚暴露跨行约束；（5）对话里最有价值的不是公式，而是 Bo 反复要求用可验证构造而非逆向凑答案；（6）算法题解必须诚实区分正确结果、例子验证、证明草图与完整证明。

参考：LeetCode 621 Task Scheduler（https://leetcode.com/problems/task-scheduler/）；同会话的推导、用户给出的 ARC-AGI-3 工程 Debug 笔记（仅作为叙事与排版风格参考）。

17. 附录 A：从「我说的是 typo」到反复重排——非数学错误，也必须保留

这一段与最优公式的推导无直接关系，却是本次 Debug 记录不可删的一部分：讨论一个例子时，如果把一次明显的打字错误误当作算法漏洞，后续所有最优性讨论都会建立在假数据上。

18.1 原意、误读、再误读

「你看不出我是 type of error 吗？」

当时要讨论的是 A×3、B/C/D/E 各×2，n=2。Bo 一直想在固定 A
间隔的构造里观察为何会产生尾部等待。中途某个位置把 A 写成
D，助手没有结合事先声明的计数检查，反而断言 D 出现三次；随后又一次次忽略「E 与 E
中间至少两个槽」的限制。

环节	当时说法	审计后的结论
原始例子	A×3；B/C/D/E 各×2	任务总数 N=11
手写串出现 D 多一次	助手按原样据此否定思路	必须先核对是否明显笔误，而不是立即改变命题
E 在 10 与 12	“ 长度 12，可以 ”	两个 E 之间仅 1 槽：不合法
E 在 10 与 13	合法，长度 13	两个 E 之间有 2 槽：合法
改成长度 12 的另一个排列	助手替用户改布局	没有回应用户要研究的固定构造过程

```
原意最终被澄清为：
位置    1  2  3  4  5  6  7  8  9  10  11  12  13
任务    A  B  C  D  A  B  C  D  A  E   _   _   E
核对：A=3,  B=2,  C=2,  D=2,  E=2;  n=2;  长度=13。
```

这个过程的 “ bug ” 不是 C++ 或贪心算法，而是协作方没有先把输入条件与 task
计数统一。即使最后给出正确公式，这种 Debug 错误也会直接降低整个讨论的有效性。

18.2 原本想对比的，是固定 A 间隔而非求现成答案

Bo 要的是同一批任务、同一个 n，比较 A 的实际间隔取 2 与取 3
时，固定式插空构造如何变化。助手却多次直接展示 “ 最优
11 ” 的排列，把探索问题切换成验证答案。这不是数学上的反例，而是任务目标被悄悄改掉。

```
n  始终为  2；这是允许的最小间隔。
固定  A  每次恰隔两个槽：A  _  _  A  _  _  A
选择  A  每次隔三个槽：A  _  _  _  A  _  _  _  A
两者都合法；问题是其余  B,C,D,E  怎么填、会产生多少  idle。
```

13 槽构造是一个合法的候选解；11 槽排列给了上界见证。不能从两者并存，直接推导某个一般性的选择定理。

18. 附录 B：五行三列（5 rounds × 3 slots）逐格审计

这个例子来自 k>n 的讨论。初始 A/B/C/D/E 都各 3 次，n=2，L=n+1=3。Bo 要求：“ 还是按照 n+1
来排每行 ”“ 相同任务首先隔行对齐 ”。此处 “ 对齐 ” 的准确操作是：相邻两行都存在某个
task，就将它放在相同列。

```
                列1    列2    列3
Round  1        A      B      C
Round  2        D      E      C
Round  3        D      E      A
Round  4        B      C      A
Round  5        B      D      E
顺序：A  B  C  |  D  E  C  |  D  E  A  |  B  C  A  |  B  D  E
```

任务	实际位置	连续位置差	检查
A	1, 9, 12	8, 3	合法
B	2, 10, 13	8, 3	合法
C	3, 6, 11	3, 5	合法
D	4, 7, 14	3, 7	合法
E	5, 8, 15	3, 7	合法

全部出现次数都是 3，最小位置差为 3，恰好满足 n=2。这里的 Round 1→2 的 C 同列，Round 2→3 的 D/E 同列，Round 3→4 的 A 同列，Round 4→5 的 B 同列。其他任务至少隔完整一行，即使不保持同列也符合冷却。

但是，这五行只是一个完整的见证。它展示“ k 大于行宽仍有可能用同一套分行规则排满 N ”，并不能单凭一张 5×3 的图证明所有输入都能这样排。记录中需要明确这一区别，而不是把例子直接拔高成普遍定理。

19.1 「隔行」与「相邻行」是两个不同命题

L=3 时，相邻两行同列重复出现的时间差为 3。若跳过一整行（第 r 行之后第 r+2 行），前次处于前行最后一列、下次处于后行第一列，位置差为 4，依然大于等于 3。所以真正需要主动对齐的是相邻两行的公共任务。

Bo 在讨论中反复强调过这一点。助手曾经要求额外证明“ 隔行出现也合法 ”，但其实只要清楚相对位置，直接按最小差 4 计算即可。

19. 附录 C：把长对话压缩成「主张—证据—纠正」日志

Bo 的主张/质疑	助手当时的问题	讨论后留下什么
一个小顶堆按 avail 选任务是否可行？	先用错误例子否定	更严格地区分 eligibility 和 preference
U_i 一直按频次排序	多次忽略下标含义	先固定符号定义再质疑
x 表示原构造剩余容量	一味说“ 未证最优 ”	先说清 x 是列容量还是时间槽

Bo 的主张/质疑	助手当时的问题	讨论后留下什么
alignment 太繁琐	反复建议换方法但无有效内容	难点是可行性证明，不是重复描述困难
要求尽量不泄露答案	提前给出公式	需要遵守 Mock 的提示边界
举例子帮助推导	给了错误合法性检查	必须逐项核对计数、位置与 n
五行三列、相邻同列	先说多行不兼容，后撤回	局部合法性可以形式化为位置差
有 idle 的 top-k 保护讨论	拿 no-idle 例子反驳临时置换猜想	后来改为 tie-breaking 保留原始 top-k
统一一套构造规则	把 k<n 与 k>n 说成不同算法	选择规则相同，演化状态不同

这段记录的现实意义是：否定数学命题需要满足前提的反例；一个合法样例只是可行性见证，不是一般性证明。协作反馈需要对齐已声明的变量、适用条件和需要证明的结论。

20. 附录 D：统一分行构造的「可执行规格」与待证条件

为了方便日后继续证明，这里把 Bo 的算法想法写成一份不冒充既有完整证明的规格：输入 task 频次、冷却 n；每轮优先取剩余最多的不同任务，限制单轮 L=n+1
个；同频决定可选集合后，相邻轮的公共任务保持同列；填入其他任务并更新频次。

调度不变量（已经可以逐项验证）：

I1. 当前行同一 task 不重复。

I2. 相邻行的公共 task 在同列。

I3. 跨过至少一整行再出现的 task 自动满足冷却。

I4. 每执行一个 task，准确将该类型 remaining 减一。

尚需额外严格证明的性质：

P1. 对所有输入，选出的下一行 task 集合都能与之前的列约束兼容。

P2. 当 N 主导时，不会提前陷入被迫 idle。

P3. 当冷却骨架主导时，所有额外 idle 都是不可避免的。

不变量 I1 – I4 是具体可检测的。P1 – P3 才涉及统一构造的一般性最优性。这样分开后，既保留 Bo 已经推进的结果，也避免把一张表格或一个成功案例误写为完整证明。

引用说明与来源索引｜LC 621 Task Scheduler

引用版 V8 · 原 V7 的 13 页正文保持不变；本页为正文观点建立来源对应关系。

引用分类

【P】原始问题与约束：LeetCode 621 官方题面 [1]。

【S】标准最优时间公式及 Heap / Greedy 方法：NeetCode 621 题解 [2]；LeetCode Wiki [3]。

【D】U1 骨架、x 容量、不变量探索、alignment 思维吸引子、逐行构造

与反复纠错：来自本次 Bo Hou 与 ChatGPT 的技术推导对话（第一手讨论记录）。

【A】形式化证明的充分性：与公式数值正确是两个层次。

本文如标注“ 尚待证明 ”，应理解为研究过程中的开放证明义务，而非已证定理。

正文阅读时的引用规则

问题定义、冷却长度及输入限制 → [1]；最终公式与标准方法 → [2][3]。

独立推导过程及助手误判 → [D]，属于作者的对话记录，不引用外部题解冒充来源。

一切实例排列需独立核验任务频次、位置间距与总长度；不能靠公式反推实例。

参考文献

[1] LeetCode. 621. Task Scheduler. 官方题目与 Hint.
https://leetcode.com/problems/task-scheduler/

[2] NeetCode. Task Scheduler — Explanation, Greedy and Math.
https://neetcode.io/solutions/task-scheduler

[3] LeetCode Wiki (Doocs). 621. Task Scheduler.
https://leetcode.doocs.org/en/lc/621/

引用说明：在线来源可更新；本文核对日期 2026-10-25。